Rockchip HDCP 开发指南

文件标识：RK-KF-YF-768

发布版本：V2.0.0

日期：2026-03-18

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按"现状"提供，瑞芯微电子股份有限公司（"本公司"，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

"Rockchip"、"瑞芯微"、"瑞芯"均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2023 ~ 2026 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

本文档主要介绍 Rockchip RK3588、RK3576、RK3572、RK3538、RK3539 平台 HDMITX / HDMIRX / DPTX 的 HDCP1.4 和 HDCP2.3 配置说明、固件打包、常见问题分析等。

产品版本

芯片名称	Android 版本支持	Linux SDK 版本支持
RK3588	Android 12 及以上	Linux 5.10 及以上
RK3576	Android 14 及以上	Linux 6.1 及以上
RK3572	Android 14 及以上	Linux 6.12
RK3538	Android 14 及以上	Linux 6.1
RK3539	Android 14 及以上	Linux 6.1 及以上

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	陈顺庆、操瑞杰、张玉炳	2023-07-21	初始发布
V2.0.0	张嘉津	2026-03-18	1. 重命名文档以支持多平台 2. 增加 RK3576/3572/3538/3539 支持 3. 完善 HDCP2.3 配置及说明 4. 完善 HDCP 验证说明等

目录

Rockchip HDCP 开发指南
- 1. HDCP 概述
  - 1.1 控制器端口分配
- 2. HDCP 关键概念说明
  - 2.1 关键模块
  - 2.2 HDCP1.4 与 HDCP2.3 差异
  - 2.3 密钥类型
- 3. HDCP Key 结构与烧写
  - 3.1 HDCP1.4 Key 结构
  - 3.2 HDCP2.3 Key 烧写模式
- 4. HDCP 工具说明
  - 4.1 RKDevInfoTool 图形化工具（Windows）
  - 4.2 KeyConvertor 工具（Windows）
  - 4.3 加密烧写库 `libhdcp.so`

4.4 RKIV 加密（可选项，非必须）
5. HDCP1.4 配置及说明
5.1 HDMIRX
5.1.1 DTS 配置
5.1.2 Key 烧写
5.1.2.1 RK 工具烧写
5.1.2.2 客户自行开发工具烧写
5.2 HDMITX
5.2.1 Key 烧写
5.3 DPTX
5.3.1 Key 烧写
6. HDCP2.3 配置及说明
6.1 HDCP2.3 DTS 配置
6.2 Firmware 固件编译与打包
6.2.1 固件编译
6.2.2 固件打包
6.3 HDMI RX Key 烧写
6.4 应用服务部署
6.4.1 服务参数说明
6.4.2 Android 配置
6.4.3 Linux 配置
7. HDCP 状态查看与验证
7.1 HDMIRX
7.2 HDMITX / DPTX 状态
7.3 Modetest 验证示例
8. HDCP2.3 常见问题分析
8.1 hdcp2_tx_rx 启动失败
8.1.1 Firmware 未导入
8.1.2 HDCP 驱动未加载
8.1.3 无报错直接退出
8.1.4 RX Key 不对
8.2 启动服务时 log 显示 `hostlib_init_mem failed`
8.3 HDMIRX 黑屏/绿屏，插拔无法恢复
8.4 同时开启 HDMITX + HDMIRX 的 HDCP2.3，拔出 RX 时 TX 闪屏
8.5 提示 vendor_storage_init failed ret=-19

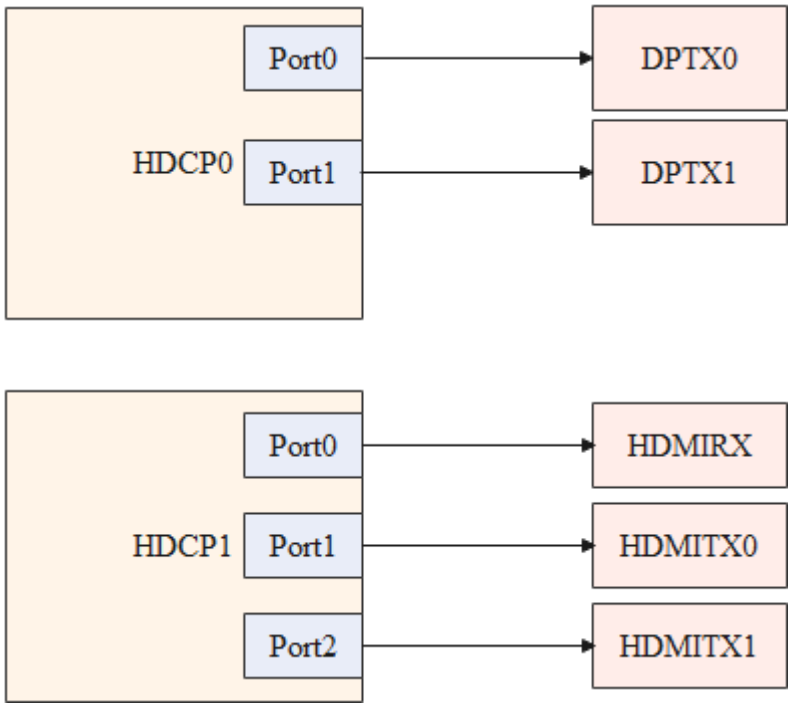
1. HDCP 概述

HDCP（High-bandwidth Digital Content Protection）是由 Intel 开发的数字内容保护技术，用于防止高清音视频内容在传输过程中被非法复制。HDCP 通过在发送端和接收端之间建立加密链路，确保只有经过认证的设备才能正确解密和播放受保护的内容。

目前 HDCP 主要有两个版本：HDCP1.4 和 HDCP2.3，其中 HDCP2.3 提供更强的加密算法和更完善的认证机制。

- RK3588、RK3576、RK3572、RK3538、RK3539 均支持 HDCP1.4 和 HDCP2.3，但 HDCP2.3 控制器端口分配不同。
- **HDCP1.4 / HDCP2.3 Key 均需客户自行向 HDCP 官网购买，芯片未内置 Key。**
- **HDCP 认证前需自测功能是否正常，是否已烧录 HDCP Key。**

1.1 控制器端口分配



RK3588 平台有两个独立的 HDCP2.3 控制器 (hdcp0 和 hdcp1)，端口示意图参考以上，每个 HDCP2.3 控制器都有 3 个 Port：

- hdcp0：DPTX0 和 DPTX1
- hdcp1：HDMIRX、HDMITX0 和 HDMITX1

HDCP Port 对应参考以下：

芯片	控制器	端口	功能
RK3588	HDCP0	port0	DPTX0
	HDCP0	port1	DPTX1
	HDCP1	port0	HDMIRX
	HDCP1	port1	HDMITX0
	HDCP1	port2	HDMITX1
RK3576	HDCP0	port1	HDMITX
	HDCP1	port0	DPTX
RK3572	HDCP0	port1	HDMITX
RK3538	HDCP0	port1	HDMITX
RK3539	HDCP0	port1	HDMITX

注意：

- **RK3576 不支持 HDMIRX**，其 DPTX 使用 HDCP1 port0，HDMITX 使用 HDCP0 port1
- **RK3572/3538/3539 不支持 HDMIRX 和 DPTX**，仅支持 HDMITX，使用 HDCP0 port1

Port 概念说明：

- **HDCP Port** 是 HDCP 控制器的硬件接口端口，用于 HDCP 认证链路的建立，每个 Port 对应一个物理显示接口（HDMITX/HDMIRX/DPTX）
- **Video Port** 是显示子系统的数据流通道，用于视频帧数据的传输和路由，两者是不同的概念，编号体系独立，无关联

2. HDCP 关键概念说明

2.1 关键模块

HDCP 加密功能涉及三个关键模块：**HDCP Key**、**Firmware**、**Service**。

模块	作用	存储位置
HDCP Key	HDCP 认证核心密钥，用于设备身份验证和内容加密	Vendor Storage（单独烧录）或 Firmware 固件（打包编译）
Firmware	HDCP2.3 控制器的固件程序，包含认证协议实现和 HDCP Key	`/vendor/firmware/`（Android）或 `/usr/lib/firmware/`（Linux）
Service	HDCP2.3 用户态服务，负责加载 Firmware、管理认证流程	`/vendor/bin/`（Android）或 `/usr/bin/`（Linux），需开机自启

2.2 HDCP1.4 与 HDCP2.3 差异

对比项	HDCP1.4	HDCP2.3
HDCP Key	所有 Key 需单独烧录到 Vendor Storage	TX Key 打包到 Firmware，RX Key 单独烧录到 Vendor Storage
Firmware	无需 Firmware	必须部署 `hdcp2_hdmi.fw` 或 `hdcp2_dp.fw`
Service	驱动内置处理，无需独立服务	必须运行 `hdcp2_tx_rx` 服务并配置开机自启
配置复杂度	相对简单，仅需烧录 Key 和配置 DTS	需完成 DTS 配置、固件编译打包、服务部署等多个步骤

说明：

- HDCP1.4 由驱动直接处理认证流程，客户只需烧录 HDCP Key 即可使用
- HDCP2.3 需要用户态服务参与，用于加载固件、处理密钥交换协议、管理认证状态等
- 使能 HDCP2.3 时会自动支持 HDCP1.4 兼容模式，无需单独配置

2.3 密钥类型

HDCP 相关密钥分为认证密钥和加密密钥两类：

密钥类型	分类	来源	存储位置
HDCP Key	认证密钥	HDCP 官网购买	Vendor Storage / Firmware
PKF / DUK	加密密钥	Rockchip 出厂预置	OTP
RKIV	加密密钥	客户自定义	Vendor Storage

密钥用途说明：

密钥类型	用途说明
HDCP Key	HDCP 认证核心密钥，用于建立加密链路
PKF / DUK	Rockchip 用于保护 HDCP2.3 Firmware 的默认加密密钥，RK3588 出厂预置在 OTP，其他芯片 OTP 默认值为全 0，客户可根据需求进行修改
RKIV	可选 AES 加密密钥（16 Bytes），用于 Firmware 打包时加密，增加客户固件唯一性

3. HDCP Key 结构与烧写

3.1 HDCP1.4 Key 结构

HDCP1.4 Key 共 **308 Bytes**，结构如下：

字段	长度	说明
KSV	8 Bytes	5 Bytes KSV + 3 Bytes 0x0
DPK	280 Bytes	核心密钥，SEED 加密仅对此部分进行
SHA	20 Bytes	校验值

注意：HDCP1.4 的 Key 需要用工具单独烧录

3.2 HDCP2.3 Key 烧写模式

HDCP2.3 的 Key 烧写模式：

类型	烧写方式	说明
HDMITX / DPTX TX Key	打包到 Firmware 固件中	TX Key 在编译固件时打包，部署固件即可
HDMIRX RX Key	打包到 Firmware 或单独烧写 RX Key	见下方说明

HDMIRX RX Key 烧写方式：

RX Key 有两种处理方式，根据量产需求选择：

方式	说明	适用场景
打包到 Firmware	RX Key 在编译时打包到 Firmware 固件中	单台设备开发调试
单独烧录	使用 RKDevInfoTool 或 libhdcp.so 将 Key 烧写到 Vendor Storage	量产场景，多台设备使用

方式一（打包到 Firmware）限制：

- Firmware 固件只能携带一个 RX HDCP Key，因此只能用于一台设备，仅适用于开发调试或单台设备使用场景
- 不允许多台设备使用相同的 RX HDCP Key

方式二（单独烧录）说明：

工具	文件格式	说明
RKDevInfoTool	`hdcprxKeys*.skf`	使用 RK 提供的图形化工具烧写
libhdcp.so	`fw_hdcp_receivers_*.bin`	客户自行开发烧写工具

注意：

- HDCP2.3 的 TX Key 已打包到 Firmware 固件中，部署固件即可
- 量产产品建议使用方式二（单独烧录），每台设备烧录不同的 RX Key
- Firmware 固件必须部署到设备

4. HDCP 工具说明

4.1 RKDevInfoTool 图形化工具（Windows）

- **SDK 存放路径：**
  - Android： `RKTools/windows/RKDevInfoWriteTool-v1.4.0`
  - Linux： `tools/windows/RKDevInfoWriteTool-v1.4.0`
- **适用范围：**
  - HDCP1.4：HDMITX / HDMIRX / DPTX Key 烧写
  - HDCP2.3：HDMIRX RX Key 烧写
- 支持 **AES + SEED 双重加密**，写入 Vendor Storage。

说明：工具中的 `RK3588` 选项用于 RK3538/RK3572/RK3576/RK3588 等 HDMI 驱动使用 dw-hdmi-qp.c 的芯片。

4.2 KeyConvertor 工具（Windows）

Key 拆分转换工具 `RKTools/windows/KeyConvertor-V2.0.7`，支持将原始 Key 拆分为单 Key 文件或 `.skf` 格式。

为什么需要拆分转换：

客户从 HDCP 官网购买的 Key 文件通常是一个文件包含多个 Key（例如购买 100 个 Key，原始文件中包含 100 个 Key 的数据），但实际烧写时：

场景	格式要求	原因
RKDevInfoTool 烧写	`.skf` 格式	RK 工具专用格式，包含加密和校验信息
libhdcp.so 自定义工具烧写	单独 bin 文件（每文件一个 Key）	每台设备需烧写不同的 Key，需单独提取
Firmware 固件编译（HDCP2.3）	`hdcp_transmitter.bin` / `hdcp_receivers.bin`	固件打包需要单个 Key 文件

4.3 加密烧写库 `libhdcp.so`

仅支持 SEED 加密，客户可自行开发烧写工具。需通过 Redmine 或 FAE 获取。

适用范围：

- HDCP1.4：HDMITX / HDMIRX / DPTX Key 烧写
- HDCP2.3：HDMIRX RX Key 烧写

接口说明：

```
enum HDCP_KEY_ID {
    HDCP1X_KEY_HDMITX_RK33 = 0,
    HDCP1X_KEY_HDMITX_RK3588,
    HDCP1X_KEY_HDMIRX_RK3588,
    HDCP1X_KEY_HDMIRX_RK628,
    HDCP1X_KEY_DP_RK3588,
    HDCP2X_KEY_HDMIRX_RK3588,
    HDCP2X_KEY_RKIV_RK3588,
};

/*
 * Encrypt the key, but is not written to vendor
 *
 * id: HDCP KEY type
 * keyin: HDCP1.x 308 Byte raw Key, HDCP2.x 1000 Byte Key
 * keyin_size: HDCP1.x 308, HDCP2.x 1000
 * keyout: The encrypted key
 * keyout_size: HDCP1.x 314, HDCP2.x 1000
 */
int hdcp_key_process(enum HDCP_KEY_ID id, uint8_t *keyin, int keyin_size, uint8_t *keyout,
int keyout_size);

/*
 * Encrypt key and write it to vendor
 *
 * id: HDCP KEY type
```

```
 * keyin: HDCP1.x 308 Byte raw Key, HDCP2.x 1000 Byte Key
 * keyin_size: HDCP1.x 308, HDCP2.x 1000
 */
int hdcp_key_process_and_write(enum HDCP_KEY_ID id, uint8_t *keyin, int keyin_size);

/*
 * Write key to vendor
 *
 * id: HDCP KEY type
 * key: HDCP1.x 308 Byte raw Key, HDCP2.x 1000 Byte Key
 * size: key size
 */
int hdcp_key_write(enum HDCP_KEY_ID id, uint8_t *key, int size);

/*
 * Read key from vendor
 *
 * id: HDCP KEY type
 * key: HDCP1.x 314 Byte Key, HDCP2.x 1000 Byte Key
 * len: key length
 */
int hdcp_key_read(enum HDCP_KEY_ID id, uint8_t *key, int *len);

/*
 * Write rkiv to vendor
 *
 * rkiv: rkiv
 * len: must be 16
 */
int hdcp_rkiv_write(uint8_t *rkiv, int len);
```

4.4 RKIV 加密（可选项，非必须）

Firmware 和 RX Key 默认会有 PKF / DUK 加密，然后在 HDCP 控制器 load Firmware 和 RX Key 之后用 PKF / DUK 进行解密。

为了确保每个客户 Firmware 的唯一性，可以在 Firmware 打包时用密钥 RKIV 进行加密。HDCP2 服务 load Firmware 之后会先进行 RKIV 解密，再传给 HDCP 控制器进行 PKF / DUK 解密。

密钥加密保护流程：

```
原始 HDCP Key（Firmware 或 RX Key）
        ↓
    [PKF/DUK 加密] ← 默认，RK3588 出厂预置
        ↓
    [RKIV AES 加密] ← 可选，Firmware 打包时加密
        ↓
    存储到设备（固件 或 Vendor Storage）
        ↓
    HDCP 服务加载时逐层解密
```

配置步骤：

1. 修改 `firmware.aic.nouart`，配置 16 bytes 的 RKIV，RKIV 为加密密钥，可自行定义，但长度必须是 16 bytes：

```
--- a/DWC_HDCP2_BASE_ESM_Firmware/firmware/firmware.aic.nouart
+++ b/DWC_HDCP2_BASE_ESM_Firmware/firmware/firmware.aic.nouart
@@ -2,6 +2,7 @@ PKF      = 0x00112233445566778899aabbccddeeff
 DUK      = 0xffeeddccbbaa99887766554433221100
 IK       = 0xffeeddccbbaa99887766554433221100
 IVc      = 0xdeadbeeffedcba9876543210
+RKIV     = 0x00112233445566778899aabbccddeeff
 BBRcode = 6144
 BBRdwb   = 3072
 CLL      = 64
```

2. 调用 libhdcp.so 库接口，把 RKIV 烧录到 vendor storage：

```
int main(void)
{
    uint8_t rkiv[] = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55, 0x66, 0x77,
                      0x88, 0x99, 0xaa, 0xbb, 0xcc, 0xdd, 0xee, 0xff};

    hdcp_rkiv_write(rkiv, 16);

    return 0;
}
```

3. 烧录 RX Key（RKIV 和 RX Key 存储在同一个 Vendor Storage 分区，烧录 RKIV 后会覆盖该分区，因此必须重新烧录 RX Key）
4. 使用 RKIV 加密，需要保证 `hdcp2_tx_rx` 在 V3.4 及以上版本

验证版本：

```
rk3588_t:/ # logcat | grep HDCP2
03-16 07:36:03.988   347   347 D HDCP2   : hdcp2 version v3.4
03-16 07:36:03.988   347   347 D HDCP2   : HDCP Port: 1
03-16 07:36:03.991   347   347 D HDCP2   : Firmware: [1]
```

5. HDCP1.4 配置及说明

5.1 HDMIRX

5.1.1 DTS 配置

单独使能 HDCP1.4：

```
&hdmirx_ctrler {
    status = "okay";
    hdcp1x-enable;
};
```

说明：如果已经使能 HDCP2.3，会默认使能 HDCP1.4，无需单独配置 HDCP1.4

5.1.2 Key 烧写

5.1.2.1 RK 工具烧写

Key 拆分和转换工具从 SDK 获取：

```
RKTools/windows/Rockchip_HdcpKey_Writer_V1.0.1.7z
```

工具包下面有 `KeyConvertor`，更新到 `KeyConvertor-V2.0.7` 可以拆分成一个 bin 文件只包含一个 Key。

Key 烧写步骤：

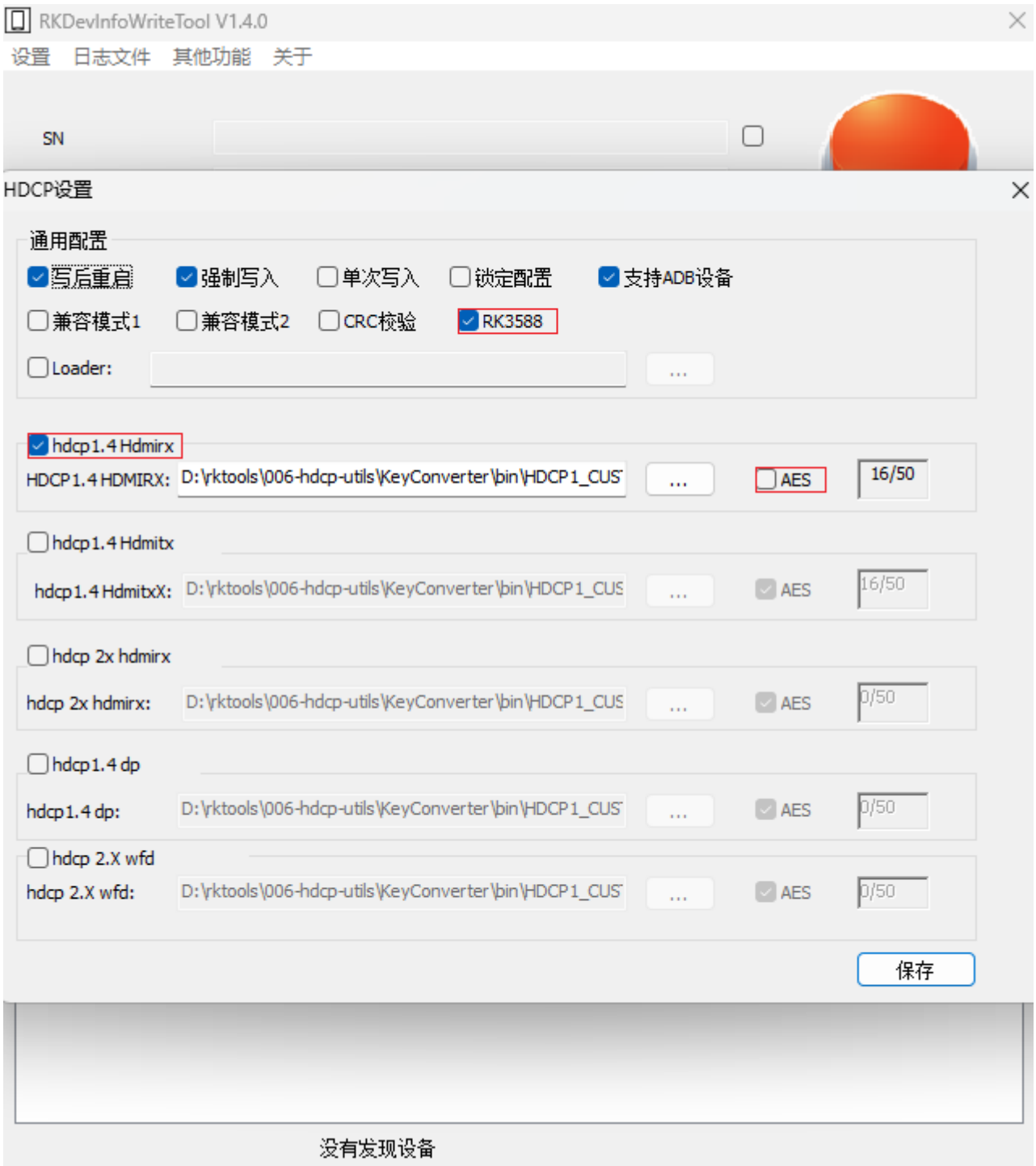
1. 先用 Key 拆分和转换工具，从原始的 Key 文件中拆出部分 Key，然后把该 Key 转成 `.skf` 后缀的烧写 Key。
2. 如果不用 RK 提供的工具烧写，需要把 Key 拆成一个文件只包含一个 Key，可以在提取原始 Key 的"每文件 Key 个数"填入 1，点提取即可生成单独的 Key 文件，不需要转成 `.skf` 格式。



3. 机器进入 Loader 模式，打开 `RKDevInfoTool` 点击 `设置` 和 `HDCPKey设置` 勾选以下选项并设置 Key 路径：
    - `hdcp1.4Hdmirx`

- `RK3588`（按需，参考《RKDevInfoTool 图形化工具》说明）
- `AES`（可选项，不选则只有 SEED 加密，选上则在 SEED 加密基础上再进行 AES 加密，更安全）

4. 导入 `.skf` 文件，点击写入即可完成 Key 的烧写。



说明：工具写完一个 Key 之后自动会跳到下一个 Key。

5.1.2.2 客户自行开发工具烧写

可以自己写应用，调用 `libhdcp.so` 接口，对 Key 进行加密并烧写到 Vendor Storage。

5.2 HDMITX

HDMITX HDCP1.4 不需要特别配置 DTS，驱动默认注册。

5.2.1 Key 烧写

Key 拆分和转换工具与 HDMIRX 一致。

烧写 Key 步骤：

机器进入 Loader 模式，`RKDevInfoTool` 点击设置，勾选以下选项：

- `hdcp1.4 Hdmitx`
- `RK3588`（按需，参考《RKDevInfoTool 图形化工具》说明）
- `强制写入`
- `单次写入`

然后导入 Key 转换后的 `.skf` 文件，点击写入即可完成 Key 的烧写。

说明：工具写完一个 Key 之后自动会跳到下一个 Key。

5.3 DPTX

DPTX 驱动默认支持 HDCP1.4，不需要额外 DTS 配置。

5.3.1 Key 烧写

Key 拆分和转换工具与 HDMIRX 一致。

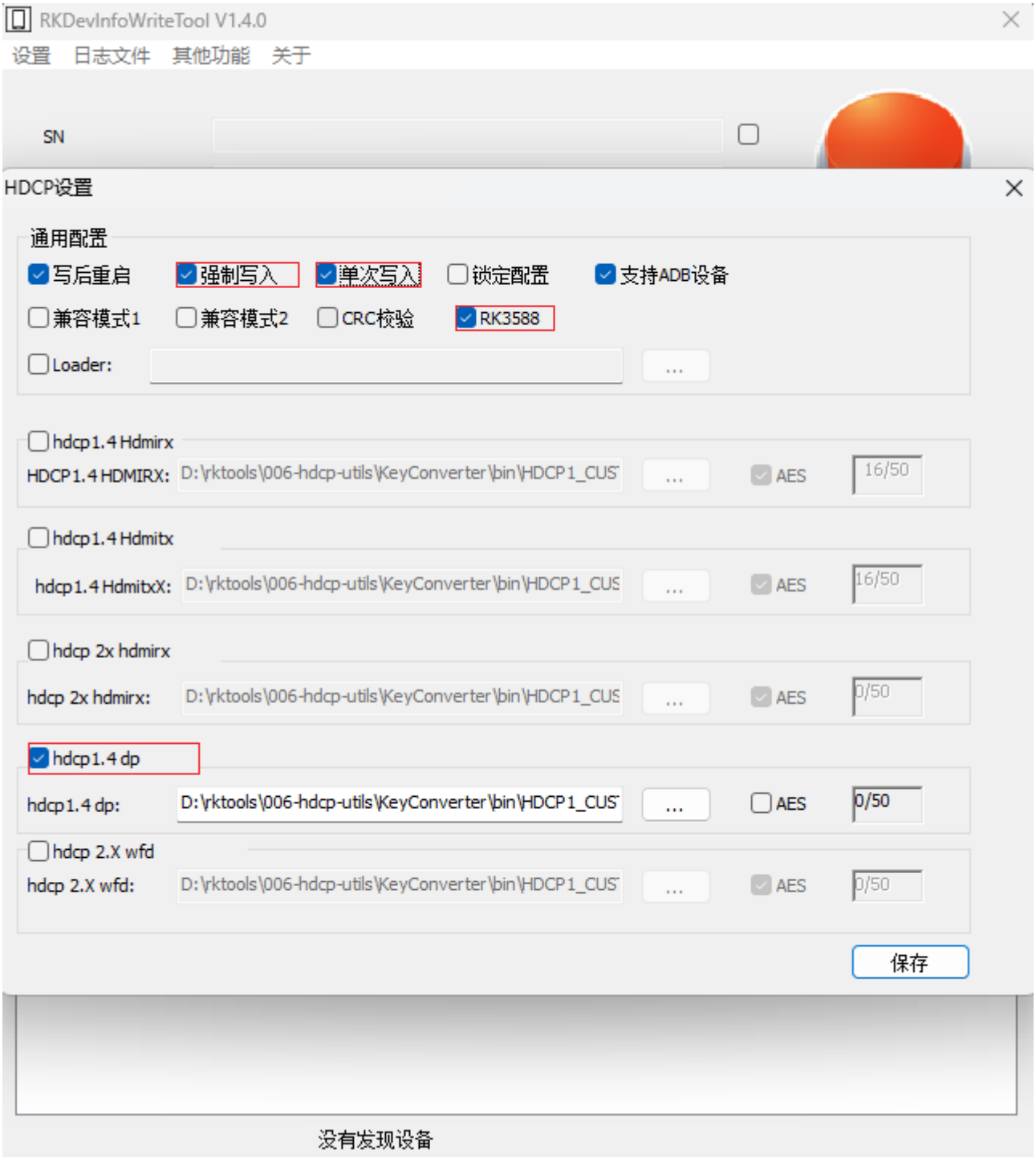
烧写 Key 步骤：

机器进入 Loader 模式，打开 `RKDevInfoTool` 点击 `设置` 和 `HDCPKey设置` 勾选以下选项并设置 Key 路径：

- `hdcp1.4 dp`
- `RK3588`（按需，参考《RKDevInfoTool 图形化工具》说明）
- `强制写入`
- `单次写入`

然后导入 Key 转换后的 `.skf` 文件，点击写入即可完成 Key 的烧写。

说明：RK3588 HDCP1.4 Key 增加了 AES 加密，`AES` 为可选项。



6. HDCP2.3 配置及说明

HDCP2.3 需完成以下步骤：

1. DTS 使能对应控制器
2. 编译 Firmware 固件（`hdcp2_hdmi.fw` / `hdcp2_dp.fw`）
3. 打包固件到目标系统
4. 烧写 HDCP2.3 RX Key（仅 HDMIRX）
5. 部署 `hdcp2_tx_rx` 服务并配置开机自启

说明：

- HDCP2.3 的 TX Key 已打包到 Firmware 固件中，只有 HDMIRX 的 RX Key 需要单独烧录。
- **Android SDK 的 `hdcp2_tx_rx` 服务默认存放在 `hardware/rockchip/hdcp2` 中，若没有需通过 Redmine 或 FAE 渠道获取。**

注意：使能 HDCP2.3 时会默认使能 HDCP1.4，无需单独配置 HDCP1.4。

6.1 HDCP2.3 DTS 配置

HDMIRX

注意：RK3588 支持 HDMI RX，RK3576 不支持 HDMI RX

```
&hdcp1 {
    status = "okay";
};

&hdmirx_ctrler {
    status = "okay";
    hdcp2x-enable;
};
```

HDMITX

RK3588 HDMITX 与 HDMIRX 共用一个 HDCP 模块：

```
&hdcp1 {
    status = "okay";
};
```

RK3576 / RK3572 / RK3538 / RK3539：

```
&hdcp0 {
    status = "okay";
};
```

注意：HDMI TX 实际 HDCP 认证版本根据 Sink 端寄存器判断

DPTX

RK3588：

```
&hdcp0 {
    status = "okay";
};
```

RK3576：

```
&hdcp1 {
    status = "okay";
};
```

注意：RK3572、RK3538 和 RK3539 不支持 DPTX。

6.2 Firmware 固件编译与打包

6.2.1 固件编译

DWC_HDCP2_BASE_ESM_Firmware 固件编译工具没有随 SDK 发布，需要通过 Redmine 或邮件向 RK 获取。

步骤 1：准备 HDCP Key 文件

根据实际需求，将对应的 HDCP Key 文件拷贝到编译工具的 `根目录` 和 `tools/` 目录下：

使用场景	所需 Key 文件
单独 HDMIRX	RX Key（重命名为 `hdcp_receivers.bin`）
单独 HDMITX	TX Key（重命名为 `hdcp_transmitter.bin`）
HDMIRX + HDMITX 同时	RX Key + TX Key（重命名如上）
DPTX	TX Key（重命名为 `hdcp_transmitter.bin`）

说明：

- 编译脚本会查找固定文件名 `hdcp_receivers.bin` 和 `hdcp_transmitter.bin`，需将实际 Key 文件重命名为此名称。
- 这里打包 RX Key 生成的 Firmware 只可用于单台设备场景，详情参考《HDCP2.3 Key 烧写模式》章节。

```
# 示例：DPTX 固件编译
DWC_HDCP2_BASE_ESM_Firmware$
cp ../hdcp_transmitter.bin .
cp ../hdcp_transmitter.bin tools/

# 示例：HDMIRX + HDMITX 固件编译
DWC_HDCP2_BASE_ESM_Firmware$
cp ../hdcp_receivers.bin .
cp ../hdcp_receivers.bin tools/
cp ../hdcp_transmitter.bin .
cp ../hdcp_transmitter.bin tools/
```

步骤 2：执行编译脚本

```
DWC_HDCP2_BASE_ESM_Firmware$ ./build_rockchip_fw.sh
```

根据提示选择固件类型：

```
1 -build firmware for both HDMIRX and HDMITX
2 -build firmware for HDMITX only
3 -build firmware for DP
Choose the fw type:
```

选择	使用场景	生成固件
1	HDMIRX 或 HDMIRX + HDMITX	`./firmware/hdcp2_hdmi.fw`
2	单独 HDMITX	`./firmware/hdcp2_hdmi.fw`
3	DPTX	`./firmware/hdcp2_dp.fw`

步骤 3：生成 RX Key 文件（仅 HDMIRX）

如果选择 `1 -build firmware for both HDMIRX and HDMITX`，打包过程会有几个步骤提示输入：

1. 需要创建 RX Key 的数量，根据需求创建，比如 10 或是 100 等：

   ```
   Create Rx Key Number:
   10
   ......
   Create 10 Keys to tools/rxKeys/hdcpkeys1-10/
   ```

   生成单独的 Key 存放到 `./rxKeys/hdcpkeys?-?/` 目录下的 `fw_hdcp_receivers_*.bin` 文件。

2. 如果需要用 RK 提供的工具烧写 Key，需要打包成 `.skf` 文件：

   ```
   Do want to pack these Keys to .skf file? [y/n]:
   y
   10 file packed to ./rxKeys/hdcpKeys1-10/hdcprxkeys1-10.skf
   ```

   如果不需要 RK 的工具烧写，这边可以选择 `n`，客户用自己的工具直接烧写 `fw_hdcp_receivers_*.bin` 文件。

6.2.2 固件打包

Android 打包

将编译生成的固件放入 `device/rockchip/$(TARGET_PRODUCT)/` 目录，修改 `device.mk`：

- HDMI 固件：

```
+USE_PRODUCT_HDCP2_HDMI_FW := $(shell test -f $(LOCAL_PATH)/hdcp2_hdmi.fw && echo true)
+ifeq ($(strip $(USE_PRODUCT_HDCP2_HDMI_FW)), true)
+    PRODUCT_COPY_FILES += \
+        $(LOCAL_PATH)/hdcp2_hdmi.fw:vendor/firmware/hdcp2_hdmi.fw
+endif
```

- DP 固件：

```
+USE_PRODUCT_HDCP2_DP_FW := $(shell test -f $(LOCAL_PATH)/hdcp2_dp.fw && echo true)
+ifeq ($(strip $(USE_PRODUCT_HDCP2_DP_FW)), true)
+    PRODUCT_COPY_FILES += \
+        $(LOCAL_PATH)/hdcp2_dp.fw:vendor/firmware/hdcp2_dp.fw
+endif
```

编译时会自动拷贝到 `/vendor/firmware/` 目录下。

Linux 打包

将编译生成的固件拷贝至目标文件系统：

固件类型	目标路径
HDMI	`/usr/lib/firmware/hdcp2_hdmi.fw`
DP	`/usr/lib/firmware/hdcp2_dp.fw`

编译时自动打包：

以 buildroot 为例，将固件放入 fs-overlay 目录，编译时自动打包到固件，`<your_board>` 根据实际硬件调整，举例如下：

```
mkdir -p buildroot/board/rockchip/<your_board>/fs-overlay/usr/lib/firmware/
cp hdcp2_hdmi.fw buildroot/board/rockchip/<your_board>/fs-overlay/usr/lib/firmware/
```

6.3 HDMI RX Key 烧写

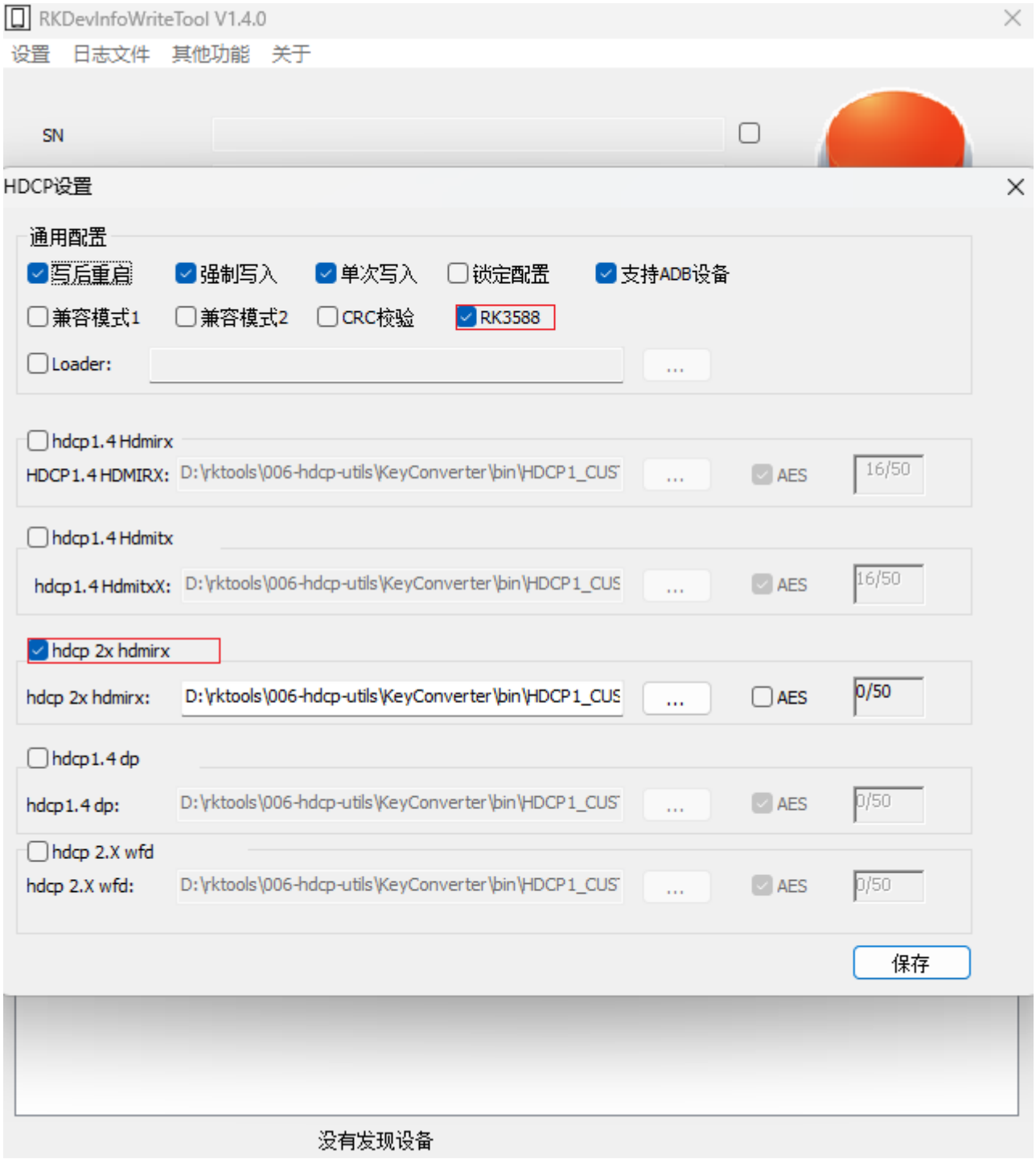
注意：RK3588 支持 HDMI RX，RK3576 不支持 HDMI RX。

HDMIRX 的 RX Key 由于每台机器需要不同的 Key，必须单独烧写。

Key 文件由 Firmware 固件编译时生成（参考上文"步骤 3：生成 RX Key 文件"），烧写方式二选一：

方式一：使用 RKDevInfoTool 工具烧写

1. 用 RKDevInfoTool.exe 工具，选择 `HDCP2X_HDMIRX`
2. 导入固件编译时生成的 `hdcprxkeys*.skf` 文件
3. 机器进入 Loader 模式，打开 `RKDevInfoTool` 点击 `设置` 和 `HDCPKey设置` 勾选下图选项并设置 Key 路径：
4. HDCP2.3 Key 不需要 AES 加密，只需勾选 `RK3588`（按需，参考《RKDevInfoTool 图形化工具》说明）和 `hdcp2x hdmirx`



说明：写完一个 Key，工具会自动跳到下一个 Key 等待烧写。

方式二：使用自定义工具烧写

1. 使用固件编译时生成的 `fw_hdcp_receivers_*.bin` 文件
2. 调用 `libhdcp.so` 库接口开发烧写工具，接口说明参考"HDCP 工具说明"章节

6.4 应用服务部署

`hdcp2_tx_rx` 服务默认存放在 SDK 目录下 `hardware/rockchip/hdcp2` 中，若没有需通过 Redmine 或 FAE 渠道获取，获取后放置到系统对应目录：

平台	目标路径
Android	`/vendor/bin/hdcp2_tx_rx`
Linux	`/usr/bin/hdcp2_tx_rx`

6.4.1 服务参数说明

详细参数说明：

```
usage: ./hdcp2_tx_rx [chip type] [hdcp] [encrypt type]
       chip type: support rk3588, rk3576, rk3572, rk3538, rk3539. (default for rk3588)
       hdcp: support 0, 1
             rk3576:
                         hdcp0 port1 for hdmitx
                         hdcp1 port0 for dptx
             rk3572/rk3538/rk3539:
                         hdcp0 port1 for hdmitx
             rk3588:
                         hdcp0 port0 for dptx0
                         hdcp0 port1 for dptx1
                         hdcp1 port0 for hdmirx
                         hdcp1 port1 for hdmitx0
                         hdcp1 port2 for hdmitx1
       encrypt type: 1, 2
                     1 for firmware encrypt
                     2 for firmware + hdmirx Key encrypt
example:
        run rk3588 hdcp0: ./hdcp2_tx_rx rk3588 0
        run rk3588 hdcp0 by default: ./hdcp2_tx_rx 0
```

参数格式：

```
hdcp2_tx_rx [芯片类型] [hdcp编号] [加密类型(可选)]
```

参数	说明
芯片类型	`rk3588`、`rk3576`、`rk3572`、`rk3538`、`rk3539`，默认 rk3588
hdcp编号	0 或 1，对应不同 HDCP 控制器
加密类型	1 = 固件加密；2 = 固件 + HDMIRX Key 加密（可选）

配置示例：

平台	功能	hdcp编号	启动命令
RK3588	DPTX0/1	hdcp0	`hdcp2_tx_rx rk3588 0`
RK3588	HDMIRX	hdcp1	`hdcp2_tx_rx rk3588 1`
RK3588	HDMITX0/1	hdcp1	`hdcp2_tx_rx rk3588 1`
RK3576	DPTX	hdcp1	`hdcp2_tx_rx rk3576 1`
RK3576	HDMITX	hdcp0	`hdcp2_tx_rx rk3576 0`
RK3572	HDMITX	hdcp0	`hdcp2_tx_rx rk3572 0`
RK3538	HDMITX	hdcp0	`hdcp2_tx_rx rk3538 0`
RK3539	HDMITX	hdcp0	`hdcp2_tx_rx rk3539 0`

6.4.2 Android 配置

修改 `hardware/rockchip/hdcp2/init.hdcp2-host.rc`，以下以 RK3588 配置 DP 服务为例：

```
diff --git a/init.hdcp2-host.rc b/init.hdcp2-host.rc
index 5a9bee8..57eef7e 100644
--- a/init.hdcp2-host.rc
+++ b/init.hdcp2-host.rc
@@ -1,7 +1,14 @@
 #on early-boot
 on post-fs
+    start hdcp2_dp
     start hdcp2_hdmi

+service hdcp2_dp /vendor/bin/hdcp2_tx_rx rk3588 0
+    class core
+    oneshot
+    disabled
+    group root system net_admin net_bt
+    capabilities DAC_OVERRIDE

 service hdcp2_hdmi /vendor/bin/hdcp2_tx_rx rk3588 1
     class core
```

系统开机时会自动加载服务。如果出现认证异常，可以通过以下命令查看 log：

```
logcat | grep HDCP2
```

6.4.3 Linux 配置

Linux 系统通过 init.d 脚本配置开机自启。

方法一：手动创建 init.d 脚本（测试单台设备）

在设备上执行以下命令创建服务脚本（将 `<平台>` 和 `<hdcp编号>` 替换为实际参数，参见上方"服务启动参数说明"）：

```
printf '%s\n' \
'#!/bin/sh' \
'case "$1" in' \
'    start)' \
'        echo "Starting HDCP2 service..."' \
'        /usr/bin/hdcp2_tx_rx <平台> <hdcp编号> &' \
'        ;;' \
'    stop)' \
'        echo "Stopping HDCP2 service..."' \
'        killall hdcp2_tx_rx' \
'        ;;' \
'    restart)' \
'        $0 stop' \
'        $0 start' \
'        ;;' \
'    *)' \
'        echo "Usage: $0 {start|stop|restart}"' \
'        exit 1' \
'        ;;' \
'esac' > /etc/init.d/S99hdcp2

# 添加执行权限
chmod +x /etc/init.d/S99hdcp2

# 测试启动
/etc/init.d/S99hdcp2 start

# 重启后自动运行
reboot
```

方法二：打包到固件（产品量产推荐）

产品量产时，将脚本集成到固件中，刷机后设备自带服务，无需逐台配置。

以 Buildroot 为例：

1. 创建 overlay 目录结构（如果没有需自行创建）：

   ```
   mkdir -p buildroot/board/rockchip/<your_board>/fs-overlay/etc/init.d/
   ```

2. 将 `S99hdcp2` 脚本放入 fs-overlay/etc/init.d 目录，脚本内容如下：

   ```
   #!/bin/sh
   case "$1" in
       start)
           echo "Starting HDCP2 service..."
           /usr/bin/hdcp2_tx_rx <平台> <hdcp编号> &
           ;;
   ```

```
    stop)
        echo "Stopping HDCP2 service..."
        killall hdcp2_tx_rx
        ;;
    restart)
        $0 stop
        $0 start
        ;;
    *)
        echo "Usage: $0 {start|stop|restart}"
        exit 1
        ;;
esac
```

说明：将 `<平台>` 和 `<hdcp编号>` 替换为实际参数，参见上方"服务启动参数说明"。

3. 添加脚本执行权限：

```
chmod +x buildroot/board/rockchip/<your_board>/fs-overlay/etc/init.d/S99hdcp2
```

4. 配置 Buildroot：

```
make menuconfig
# Rootfs (Buildroot|Debian|Yocto) -> Post rootfs installs -> Overlays
# 在 "extra overlay dirs" 中填入：board/rockchip/<your_board>/fs-overlay
```

说明：其他构建系统（如 Yocto）需要自行实现，目标都是将脚本放入文件系统的 `/etc/init.d/` 目录。

查看服务状态：

```
ps aux | grep hdcp2_tx_rx
```

7. HDCP 状态查看与验证

7.1 HDMIRX

```
cat /sys/class/misc/hdmirx_hdcp/status
```

输出内容	含义
`HDCP Disable`	HDCP 未使能
`HDCP1.4: Authenticated start`	HDCP1.4 认证过程中
`HDCP1.4: Authenticated success`	HDCP1.4 认证成功
`HDCP1.4: Authenticated failed`	HDCP1.4 认证失败
`HDCP1.4: Unknown status`	HDCP1.4 未知状态
`HDCP2.3: Authenticated success`	HDCP2.3 认证成功
`HDCP2.3: Authenticated failed`	HDCP2.3 认证失败
`HDCP2.3: No decrypted`	源端未开启 HDCP2.x 加密

Source 没有开启 HDCP 加密，或是没有接入信号源：

```
HDCP2.3: No decrypted
HDCP1.4: Unknown status
```

7.2 HDMITX / DPTX 状态

使用 `modetest` 工具快速查看对应 Connector 的属性：

```
modetest -c | grep "Content Protection" -A 15
```

关键属性说明：

属性名	类型	值含义
`Content Protection`	enum	0 = 未认证, 1 = 请求认证, 2 = 认证成功
`HDCP Content Type`	enum	0 = HDCP1.x 可传输内容, 1 = HDCP2.x 可传输内容
`hdcp_encrypted`	range	0 = 未认证, 1 = HDCP1.4, 2 = HDCP2.3

属性详解：

```
---------------------------------------------------------------------
Content Protection:
                flags: enum
                enums: Undesired=0 Desired=1 Enabled=2
                value: 2

Undesired: 关闭 HDCP
Desired: 开启 HDCP
Enabled: HDCP 已经开启并认证成功
```

```
----------------------------------------------------------------------
HDCP Content Type:
                flags: enum
                enums: HDCP Type0=0 HDCP Type1=1
                value: 0

HDCP Type0: HDCP1.4 或 HDCP2.3 任一等级认证成功，都允许输出正常画面
HDCP Type1: 必须 HDCP2.3 认证成功，才能够输出正常画面

注：该功能目前版本驱动未实际支持，默认任何等级认证成功都能输出正常，所以无需配置。
----------------------------------------------------------------------
hdcp_encrypted:
                flags: range
                values: 0 2
                value: 2

HDCP 认证等级:
    0: HDCP 未认证
    1: HDCP1.4
    2: HDCP2.3
----------------------------------------------------------------------
```

7.3 Modetest 验证示例

前提：已正确部署 HDCP2.3 固件及服务，显示器支持 HDCP

1. **确认 Connector ID**

   第一列数字即为 connector id：

   ```
   rk3576_projector:/ # modetest -c | grep connected
   411     410     connected       HDMI-A-1        1660x930                47      410
   431     430     connected       DSI-1           68x121          1       430
   433     0       disconnected    DP-1            0x0             0       432
   ```

2. **开启 HDCP**

   ```
   modetest -w 411:"Content Protection":1
   ```

3. **查看认证状态**

   ```
   modetest -c
   ```

   当对应 `Content Protection` 变为 `2` 时，表示 HDCP 认证成功：

   ```
   411     410     connected       HDMI-A-1        1660x930                47      410
     modes:
           name refresh (Hz) hdisp hss hse htot vdisp vss vse vtot)
     3840x2160 60 3840 4016 4104 4400 2160 2168 2178 2250 594000 flags: phsync, pvsync
     ...
   ```

```
  props:
        426 Content Protection:
                flags: enum
                enums: Undesired=0 Desired=1 Enabled=2
                value: 2
        427 HDCP Content Type:
                flags: enum
                enums: HDCP Type0=0 HDCP Type1=1
                value: 1
        428 hdcp_encrypted:
                flags: range
                values: 0 2
                value: 2
```

4. **关闭 HDCP**

```
modetest -w 411:"Content Protection":0
```

8. HDCP2.3 常见问题分析

8.1 hdcp2_tx_rx 启动失败

说明：以下 `hdcp2_hdmi.fw` 的路径只是举例，以实际环境为准

Android 环境查看 log：

```
logcat | grep HDCP
```

8.1.1 Firmware 未导入

```
Failed to open firmware file [vendor/firmware/hdcp2_hdmi.fw]
```

解决：检查固件是否正确放置并编译。

8.1.2 HDCP 驱动未加载

```
Failed to open firmware driver /dev/hl_dev1
```

解决：DTS 需要使能对应的 `hdcp1` 节点。

8.1.3 无报错直接退出

```
07-14 10:53:17.033  2042  2042 D HDCP2   : hdcp2 version v3.0
07-14 10:53:17.033  2042  2042 D HDCP2   : HDCP Port: 1
07-14 10:53:17.034  2042  2042 D HDCP2   : Firmware: [1]
07-14 10:53:17.034  2042  2042 D HDCP2   :           223664 bytes
07-14 10:53:17.034  2042  2042 D HDCP2   : check rx Key ok
07-14 10:53:17.034  2042  2042 D HDCP2   : rxKey read from vendor success, rxkey_len = 1000
07-14 10:53:17.034  2042  2042 D HDCP2   : Host Library version V0201
07-14 10:53:17.035  2042  2042 D HDCP2   : Exiting with err=1
```

解决：手动执行查看详细报错：

```
console:/ # ./vendor/bin/hdcp2_tx_rx 1 &
```

如果出现 `hostlib_init_mem failed`，通常是固件编译问题，可能是 Key 不对或 DUK/PKF 配置不对，Key 的编译参考 Firmware 固件编译与打包。

8.1.4 RX Key 不对

```
HLC_HDCPRX_LoadRxDwb failed
```

解决：检查 RX Key 是否正确烧写。

8.2 启动服务时 log 显示 `hostlib_init_mem failed`

现象：

```
rk3576_u:/ # hdcp2_tx_rx rk3576 1
vendor_storage_init failed ret=-19
device name = /dev/hl_dev1
...
<ERROR> [external/hdcp2_tx_rx/host_library/core_lib/host_lib_core.c:137] [PID:2302]
hostlib_init_mem failed
```

原因：固件编译时 PKF/DUK 值未正确配置，或烧录的 Key 与固件不匹配。

注意：RK3576 默认需要修改 PKF 和 DUK 为全 0。

解决：修改 `DWC_HDCP2_BASE_ESM_Firmware/firmware/firmware.aic.nouart`，将 PKF 和 DUK 全部置零：

```
diff --git a/DWC_HDCP2_BASE_ESM_Firmware/firmware/firmware.aic.nouart
b/DWC_HDCP2_BASE_ESM_Firmware/firmware/firmware.aic.nouart
index caeed33..2d113b4 100644
--- a/DWC_HDCP2_BASE_ESM_Firmware/firmware/firmware.aic.nouart
+++ b/DWC_HDCP2_BASE_ESM_Firmware/firmware/firmware.aic.nouart
@@ -1,5 +1,5 @@
-PKF     = 0x00112233445566778899aabbccddeeff
-DUK     = 0xffeeddccbbaa99887766554433221100
+PKF     = 0x00000000000000000000000000000000
+DUK     = 0x00000000000000000000000000000000
 IK      = 0xffeeddccbbaa99887766554433221100
 IVc     = 0xdeadbeeffedcba9876543210
 BBRcode = 6144
```

重新编译固件并替换后生效。

8.3 HDMIRX 黑屏/绿屏，插拔无法恢复

原因：HDCP2.3 已使能但 `hdcp2_tx_rx` 服务未运行或异常退出。

临时解决方案：

1. 确认 DTS 中 `hdcp2x-enable` 已打开
2. 检查 `hdcp2_tx_rx` 服务状态：

```
logcat | grep HDCP2
```

3. 临时关闭 HDCP 恢复画面：

```
echo 0 > /sys/class/misc/hdmirx_hdcp/enable
```

8.4 同时开启 HDMITX + HDMIRX 的 HDCP2.3，拔出 RX 时 TX 闪屏

原因：HDMITX 与 HDMIRX 共用 HDCP2.3 控制器，RX 拔出时会复位控制器，导致 TX 重新认证。

建议：避免同时使能 TX 和 RX 的 HDCP2.3。

8.5 提示 vendor_storage_init failed ret=-19

原因：Vendor Storage 分区未初始化或未烧录 HDCP Key。

解决：使用 RKDevInfoTool 烧写对应 HDCP2.3 Key（TX 或 RX），并确认写入成功。

注意：若服务被重复启动也可能触发此错误，请确保 init.rc 中服务未重复定义。